TRABAJO PRÁCTICO INTEGRADOR

GameSec Monitor: Telemetría de Servidores de Videojuegos y Detección de Incidentes con Python

Modalidad de Trabajo:	Grupal (3 a 4 estudiantes)
Tecnologías y Entorno:	Python 3.10+ (únicamente librerías estándar: `csv`, `math`, `os`). Sin Pandas/NumPy.
Entregables Obligatorios:	Código fuente (`.py`), dataset (`gamesec_logs.csv`), reporte (`reporte_gamesec.txt`) y `README.md`
Instancia de Evaluación:	Entrega en repositorio/archivo + Coloquio Oral Ágil (10 a 15 minutos por grupo)

1. Contexto de Negocio e Integración de Carreras

En la industria contemporánea del entretenimiento digital y los servicios en la nube, los servidores de videojuegos multijugador representan infraestructuras críticas donde convergen tres disciplinas fundamentales: la experiencia de juego en tiempo real (**Videojuegos**), la protección contra ataques e intrusiones (**Ciberseguridad**) y el análisis masivo de datos para toma de decisiones y modelos predictivos (**Inteligencia Artificial**).

En este Trabajo Práctico Integrador, los equipos asumirán el rol de **Desarrolladores de Software en el área de Infraestructura y Seguridad** de un estudio de videojuegos online. El desafío consiste en construir una herramienta de consola modularizada en Python capaz de procesar archivos de telemetría de servidores (logs de sesiones), calcular métricas operativas clave y clasificar automáticamente incidentes anómalos o sospechosos (posibles atacantes, bots o cheaters) mediante un motor heurístico basado en reglas de negocio.

Puntos de encuentro interdisciplinarios en el proyecto:

- **Videojuegos:** Análisis de la calidad de servicio y experiencia del jugador (latencia en `ping_promedio`, estabilidad de cuadros por segundo `fps_promedio`, duración de partidas y modalidades Casual vs. Competitiva vs. Torneo).
- **Ciberseguridad:** Auditoría de eventos anómalos (intentos fallidos de autenticación, picos de latencia intencionales conocidos como *lag switching*, y denuncias acumuladas por conducta indebida o trampas).
- **Inteligencia Artificial / Datos:** Modelado de entidades como listas de diccionarios, agregación y cálculo de estadísticas multivariables, y diseño de un clasificador por reglas como base de futuros modelos supervisados.

2. Objetivos de Aprendizaje y Alcance Técnico

Al finalizar este trabajo práctico, el equipo habrá demostrado competencia en:

1. Modelado de datos tabulares reales utilizando exclusivamente estructuras nativas: listas de diccionarios y tuplas.
2. Manejo idiomático de archivos CSV mediante el módulo estándar `csv` (lectura, tipado y validación de precondiciones).

3. Uso avanzado de diccionarios acumuladores para agrupar datos por categorías y calcular métricas derivadas.
4. Diseño de funciones puras, con parámetros formales y retornos explícitos, eliminando por completo el uso de variables globales.
5. Modularización clara del sistema en archivos `.py` coordinados, facilitando el trabajo en equipo y la defensa oral.
6. Generación de reportes consolidados en archivos de texto (`.txt`) formateados profesionalmente.

Restricción Técnica Importante

Para garantizar la asimilación profunda de las estructuras básicas de datos y el pensamiento algorítmico, **está estrictamente prohibido el uso de librerías externas de análisis** (tales como Pandas, Polars o NumPy). Toda la lógica debe implementarse con tipos nativos de Python (`dict`, `list`, `tuple`, `str`, `int`, `float`) y el módulo estándar `csv`.

3. El Conjunto de Datos: `gamesec_logs.csv`

La cátedra provee el dataset `gamesec_logs.csv`, compuesto por 150 registros representativos de sesiones de juego registradas en múltiples servidores regionales durante una ventana de monitoreo. El archivo utiliza coma (`,`) como delimitador estándar e incluye encabezados.

Columna (Campo)	Tipo de Dato	Dominio / Ejemplos	Descripción
`id_sesion`	`int`	1 a 150	Identificador único correlativo de la sesión registrada.
`servidor`	`str`	'SA-BuenosAires', 'SA-SaoPaulo', etc.	Región del servidor alojado.
`jugador_id`	`str`	Ej. 'User_125', 'User_482'	Identificador alfanumérico anonimizado del jugador.
`tipo_partida`	`str`	'Casual', 'Competitiva', 'Torneo'	Modalidad de la partida disputada.
`ping_promedio`	`int`	Milisegundos (> 0)	Latencia media de red experimentada en la sesión.
`fps_promedio`	`int`	30 a 240 fps	Rendimiento gráfico del cliente durante la partida.
`duracion_minutos`	`float`	Minutos decimales	Tiempo total de la sesión activa en el servidor.
`intentos_fallidos`	`int`	0 a 8	Intentos fallidos de conexión/autenticación previos.
`reportes_trampa`	`int`	0 a 6	Denuncias por trampas/hacks emitidas por otros participantes.

4. Arquitectura Modular del Software

Para garantizar claridad y agilidad en la defensa oral, el proyecto se organiza exactamente en tres archivos fuente con responsabilidades desacopladas y no superpuestas:

- **`carga.py`**: Contiene exclusivamente las funciones de apertura, lectura, tipado y validación de integridad del archivo CSV.
- **`procesamiento.py`**: Agrupa la lógica pura de cómputo: estadísticas generales, agrupamientos categóricos con diccionarios y el motor heurístico de clasificación de seguridad.
- **`main.py`**: Es el punto de entrada del programa. Coordina la ejecución, presenta un menú interactivo en consola y orquesta la exportación del reporte final.

5. Consignas Obligatorias de Desarrollo

Consigna 1: Módulo `carga.py` — Carga y Validación de Datos

Implementar la función encargada de leer el dataset:

```python
def cargar_telemetria(ruta_archivo: str) -> list[dict]:
    # Lee el archivo CSV con csv.DictReader
    # Convierte tipos: int (id, ping, fps, intentos, reportes), float (duracion), str (resto)
    # Retorna una lista de diccionarios
    pass
```

Requisitos específicos de la función:

1. Debe abrir el archivo usando `with open(...)` con codificación `utf-8`.
2. Utilizar `csv.DictReader` para mapear automáticamente los nombres de columna.
3. Realizar el casteo explícito de los tipos de datos numéricos (no pueden quedar como strings).
4. Validar que el archivo exista y contenga al menos un registro (lanzar `ValueError` o mensaje amigable si está vacío).

Consigna 2: Módulo `procesamiento.py` — Métricas Globales de Operación

Implementar una función que calcule un panorama estadístico general de todas las sesiones registradas:

```python
def calcular_estadisticas_globales(sesiones: list[dict]) -> dict:
    # Retorna un diccionario con las metricas consolidadas
    pass
```

El diccionario retornado debe contener exactamente las siguientes claves:

- `'total_sesiones'`: Cantidad total de registros analizados (`int`).
- `'ping_promedio'`: Latencia media global redondeada a 2 decimales (`float`).
- `'duracion_promedio'`: Duración media de partidas en minutos redondeada a 2 decimales (`float`).
- `'porcentaje_competitivas'`: Porcentaje de sesiones correspondientes a 'Competitiva' o 'Torneo' sobre el total (`float`).

Consigna 3: Módulo `procesamiento.py` — Agrupamiento por Región de Servidor

Implementar una función que aplique el patrón de diccionario acumulador para agrupar los datos por región de servidor:

```python
def analizar_por_servidor(sesiones: list[dict]) -> dict:
    # Retorna un diccionario con el formato:
    # {
    #     "SA-BuenosAires": {"cantidad": 42, "ping_promedio": 45.2, "
        fps_promedio": 115.4},
    #     ...
    # }
    pass
```

Aspectos clave a evaluar en el coloquio:

- No se deben prefijar los nombres de servidores en listas *hardcodeadas*; el diccionario debe poblarse dinámicamente a medida que aparecen servidores nuevos en el archivo.
- Deben utilizarse diccionarios acumuladores para sumarizar cantidades y totales de ping y fps.

Consigna 4: Módulo `procesamiento.py` — Motor de Clasificación de Incidentes

El núcleo de ciberseguridad del sistema consiste en evaluar cada sesión contra un conjunto de reglas determinísticas para detectar anomalías:

Reglas Heurísticas de Clasificación de Seguridad

1. **CRÍTICO:** Si la sesión registra (`reportes_trampa` $\geq$ 3) O BIEN (`intentos_fallidos` ≥ 4).
2. **SOSPECHOSO:** Si no es Crítico y cumple: [(`ping_promedio` $>$ 200 Y `intentos_fallidos` ≥ 2) O (`reportes_trampa` $== 2$)].
3. **NORMAL:** Cualquier sesión que no encaje en las categorías anteriores.

```python
def clasificar_incidentes(sesiones: list[dict]) -> tuple[dict, list[dict
    ]]:
    # Retorna una tupla:
    # 1. Diccionario resumen de frecuencias: {"Normal": N, "Sospechoso":
        S, "Critico": C}
    # 2. Lista de diccionarios simplificados correspondientes unicamente
        a los eventos 'Critico'
    pass
```

Cada elemento en la lista de eventos críticos debe ser un diccionario compacto con la siguiente estructura:

```
{
    "id_sesion": 14,
    "jugador_id": "User_482",
    "servidor": "NA-Virginia",
    "motivo": "Exceso de reportes por trampa (4) e intentos fallidos (5)
        "
}
```

Consigna 5: Módulo `main.py` — Interfaz de Consola y Reporte Final

El archivo `main.py` debe presentar un menú interactivo en consola mediante un bucle `while`:

```
========================================
      GAMESEC MONITOR - PANEL DE CONTROL
========================================
1. Cargar y validar dataset
2. Ver estadisticas globales de servidores
3. Ver rendimiento por region de servidor
4. Auditar incidentes de seguridad (Criticos y Sospechosos)
5. Exportar reporte consolidado (reporte_gamesec.txt)
6. Salir
========================================
Seleccione una opcion:
```

Asimismo, la opción 5 debe invocar a la función `exportar_reporte(sesiones, ruta_salida='reporte_ga` para escribir un informe estructurado que consolide todos los análisis realizados.

6. Estructura del Reporte de Texto: `reporte_gamesec.txt`

El archivo de texto generado automáticamente debe respetar la siguiente estructura modelo:

```
======================================================================
              REPORTE DE AUDITORIA Y TELEMETRIA     GAMESEC
======================================================================
Fecha de generacion: 05/10/2026

[1] RESUMEN GLOBAL OPERATIVO
----------------------------------------------------------------------
Total de sesiones procesadas: 150
Ping promedio global: 94.32 ms
Duracion media de partida: 28.65 min
Partidas de alta competencia (Ranked/Torneo): 48.00%

[2] RENDIMIENTO REGIONAL POR SERVIDOR
----------------------------------------------------------------------
Servidor           | Sesiones | Ping Promedio | FPS Promedio
----------------------------------------------------------------------
SA-BuenosAires     | 42       | 48.20 ms      | 112.50 fps
SA-SaoPaulo        | 38       | 54.10 ms      | 108.20 fps
NA-Virginia        | 40       | 152.40 ms     | 98.40 fps
EU-Frankfurt       | 30       | 185.60 ms     | 92.10 fps

[3] AUDITORIA DE SEGURIDAD E INCIDENTES
----------------------------------------------------------------------
Distribucion de estados de seguridad:
      Normales:    118 sesiones (78.67%)
      Sospechosos:  22 sesiones (14.67%)
      Criticos:     10 sesiones ( 6.67%)

Detalle de Sesiones Criticas Detectadas:
  [!] Sesion #14  | Jugador: User_482 | Servidor: NA-Virginia
      Motivo: Exceso de reportes por trampa (4) e intentos fallidos (5)
  [!] Sesion #29  | Jugador: User_901 | Servidor: SA-BuenosAires
      Motivo: Ataque de fuerza bruta en login (6 intentos fallidos)
  ...

[4] CONCLUSIONES TECNICAS DEL EQUIPO
----------------------------------------------------------------------
```

```
(Espacio para 3 conclusiones redactadas obligatoriamente por el grupo)
1. ...
2. ...
3. ...
=====================================================================
                    FIN DEL REPORTE GAMESEC
=====================================================================
```

7. Dinámica de Defensa Oral: Coloquio de 10 a 15 Minutos

El coloquio durará entre 10 y 15 minutos por grupo. Cada integrante deberá responder preguntas individuales sobre el diseño y la autoría del código.

Dimensión Evaluada	Peso	Criterios Clave Observados en el Coloquio
1. Autoría y Dominio de Sintaxis	25 %	El estudiante explica con soltura qué hace cada instrucción de su código, cómo se abrió el CSV y cómo se castearon los tipos de datos.
2. Estructuras de Datos Nativas	25 %	Comprensión conceptual profunda del uso de listas de diccionarios, tuplas de retorno y acumulación dinámica en diccionarios.
3. Modularidad y Calidad	25 %	Invocación correcta entre módulos sin variables globales, manejo limpio de parámetros, buenas prácticas de nomenclatura y comentarios.
4. Prueba de Live-Coding en Vivo	25 %	Modificación ágil en vivo (1 a 2 minutos) propuesta por el docente (ej.: cambiar un umbral de alerta o agregar un filtro nuevo).

8. Checklist Previo a la Entrega

Antes de subir el archivo comprimido final (`.zip`), verificar minuciosamente:

El programa se ejecuta desde cero en una consola limpia mediante `'python main.py'`.

El archivo `gamesec_logs.csv` está en la misma carpeta o ruta relativa esperada.

No se utiliza ninguna librería externa (sin pandas, sin numpy).

Todas las funciones reciben argumentos formales y retornan datos con `'return'`.

El reporte `reporte_gamesec.txt` se genera correctamente con las tres conclusiones del grupo.

El archivo `README.md` contiene los nombres, DNI y carreras de todos los integrantes.